

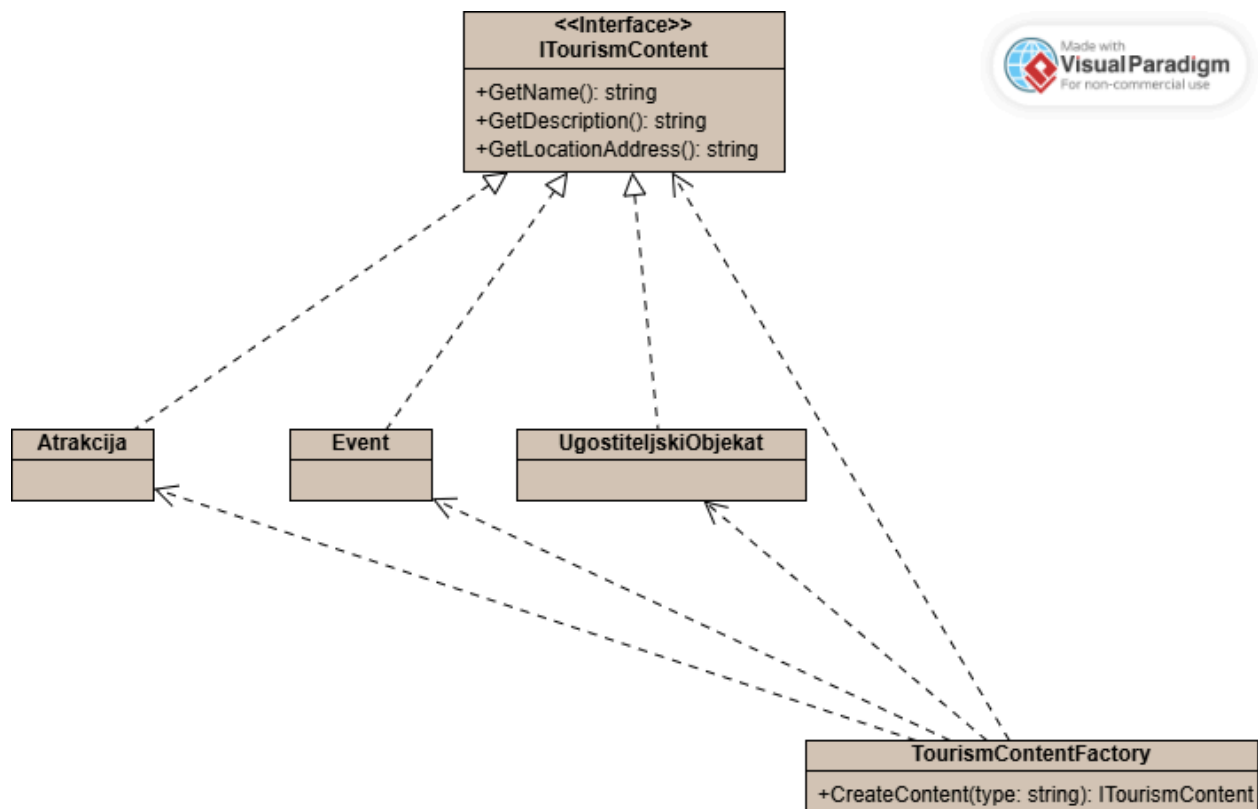
Kreacijski paterni

1. Factory Method Pattern

Factory Method pattern služi za kreiranje objekata bez direktnog specificiranja tačne klase koja će biti instancirana. Ovaj pattern definiše zajednički interfejs za kreiranje objekata, dok konkretna klasa ili factory metoda odlučuje koji tip objekta će biti kreiran. Na taj način se smanjuje zavisnost ostatka sistema od konkretnih klasa i povećava fleksibilnost aplikacije.

U sistemu **Go Sarajevo** Factory Method pattern se može primijeniti za kreiranje različitih tipova turističkog sadržaja. U aplikaciji postoje klase Attraction, Event i Hospitality, koje predstavljaju različite sadržaje dostupne korisniku. Iako se ove klase razlikuju po atributima, sve imaju zajedničku svrhu: prikazuju sadržaj koji korisnik može pregledati ili dodati u svoj plan posjete.

Zbog toga se može uvesti zajednički interfejs ITourismContent, koji bi sadržavao osnovne metode kao što su dohvat naziva, opisa i lokacije sadržaja. Klase Attraction, Event i Hospitality bi implementirale ovaj interfejs, dok bi klasa TourismContentFactory bila zadužena za kreiranje odgovarajućeg tipa sadržaja.



Na primjer, ako sistem treba kreirati atrakciju, događaj ili ugostiteljski objekat, ostatak aplikacije ne mora direktno pozivati konstruktore tih klasa. Umjesto toga, poziva se factory metoda koja na osnovu proslijeđenog tipa odlučuje koji objekat treba kreirati. Ovim pristupom se olakšava dodavanje novih tipova turističkog sadržaja u budućnosti, bez potrebe za većim izmjenama postojećeg koda.

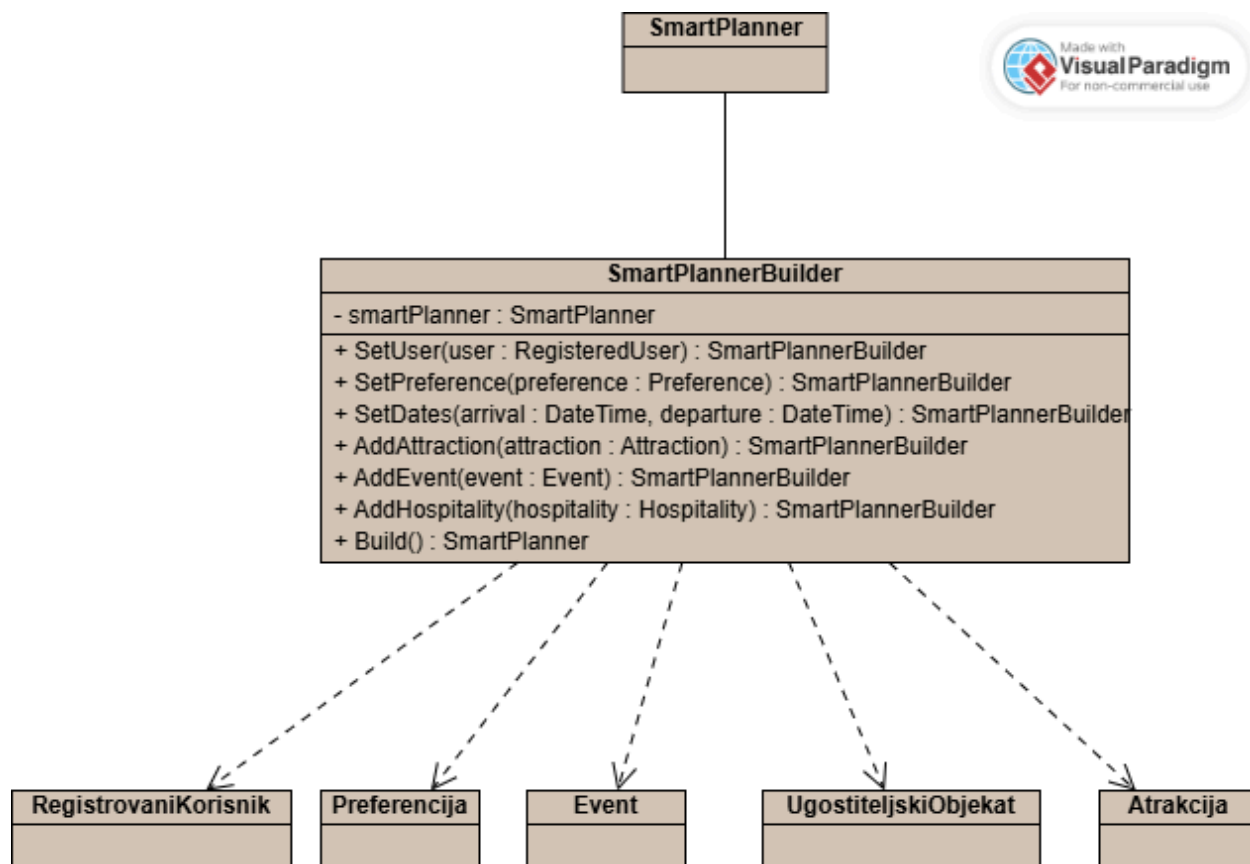
2. Builder Pattern

Builder pattern služi za kreiranje složenih objekata korak po korak. Posebno je koristan kada objekat ima više atributa, povezanih objekata ili kada njegovo kreiranje zahtijeva više faza. Umjesto da se složeni objekat kreira direktno i da se svi njegovi podaci postavljaju ručno, Builder pattern omogućava da se proces kreiranja izdvoji u posebnu klasu.

U sistemu **Go Sarajevo** Builder pattern se može primijeniti na klasu SmartPlanner. Klasa SmartPlanner predstavlja plan posjete korisnika i sastoji se od više elemenata: datuma dolaska, datuma odlaska, registrovanog korisnika, preferencija, atrakcija, događaja i ugostiteljskih objekata. Zbog toga je SmartPlanner složen objekat čije kreiranje može biti preglednije ako se izdvoji u posebnu builder klasu.

Uvođenjem klase SmartPlannerBuilder omogućava se postepeno kreiranje plana. Builder bi imao metode za postavljanje korisnika, preferencija i datuma boravka, kao i metode za dodavanje atrakcija, događaja i ugostiteljskih objekata. Na kraju procesa metoda Build() bi vraćala kompletno kreiran objekat tipa SmartPlanner.

Ovakav pristup poboljšava čitljivost i organizaciju koda, jer se proces kreiranja plana nalazi na jednom mjestu. Također smanjuje mogućnost grešaka, jer se plan može graditi kontrolisanim redoslijedom i uz jasne metode za svaki dio objekta. Builder pattern je posebno koristan u ovom sistemu jer se plan posjete može razlikovati od korisnika do korisnika, u zavisnosti od njegovih preferencija i odabranih sadržaja.



3. Singleton Pattern

Singleton pattern osigurava da određena klasa ima samo jednu instancu tokom izvršavanja aplikacije i da se toj instanci može pristupiti globalno. Ovaj pattern je koristan kada u sistemu postoji servis ili konfiguracija koja treba biti zajednička za cijelu aplikaciju.

U sistemu **Go Sarajevo** Singleton pattern bi se mogao primijeniti na servis koji komunicira sa vanjskim API servisima, na primjer GooglePlacesService. Budući da aplikacija koristi podatke o lokacijama i može sadržavati GooglePlaceId unutar klase Hospitality, servis za dohvat dodatnih informacija o lokacijama mogao bi biti centralizovan u jednoj instanci.

Na taj način bi se izbjeglo kreiranje više objekata istog servisa na različitim mjestima u aplikaciji. Jedna instanca GooglePlacesService mogla bi upravljati API ključem, konfiguracijom i komunikacijom sa Google Places servisom. Time bi sistem bio pregledniji, lakši za održavanje i sigurniji u pogledu upravljanja zajedničkim resursima.

4. Prototype Pattern

Prototype pattern omogućava kreiranje novih objekata kopiranjem već postojećih objekata. Ovaj pattern je koristan kada je kreiranje objekta složeno ili kada korisnik želi napraviti novi objekt koji je sličan već postojećem.

U sistemu **Go Sarajevo** Prototype pattern bi se mogao primijeniti na klasu SmartPlanner. Korisnik može imati već kreiran plan posjete gradu i željeti napraviti novu verziju tog plana sa manjim izmjenama, na primjer promjenom datuma, dodavanjem nove atrakcije ili uklanjanjem nekog događaja.

Umjesto da korisnik ponovo kreira kompletan plan od početka, postojeći SmartPlanner objekt mogao bi se klonirati. Nakon kloniranja korisnik bi mogao izmijeniti samo one dijelove plana koji se razlikuju. Na taj način bi se ubrzalo kreiranje sličnih planova i smanjilo ponavljanje istih koraka.

5. Abstract Factory Pattern

Abstract Factory pattern omogućava kreiranje porodica povezanih objekata bez direktnog navođenja njihovih konkretnih klasa. Ovaj pattern je koristan kada sistem treba kreirati više objekata koji pripadaju istoj kategoriji ili istoj logičkoj cjelini.

U sistemu **Go Sarajevo** Abstract Factory pattern bi se mogao primijeniti za kreiranje različitih tipova turističkih planova. Na primjer, korisnik može imati različite interese, kao što su historija, priroda, kultura ili noćni život. Na osnovu toga sistem može kreirati različite porodice sadržaja za plan posjete.